

ERP System – Full Specification & Roadmap

This doc will contain the full ERP specification. See attached README file for the detailed markdown version.

ERP Product – Full Specification & Roadmap (README Version)

ERP Product – Full Specification & Roadmap

This README is the main reference document for designing and implementing a flexible, multi-activity, multi-company ERP system. Follow the phases at the end in order to minimize errors and avoid architectural rework.

Table of Contents

- a. Overview
- b. Supported Business Types & Customization
- c. Tech Stack & Architecture (incl. Docker)
- d. Multi-Company, Users & Permissions
- e. Master Data (Core Business Data)
- f. Currencies & Units of Measure
- g. Functional Modules
- h. Costing Models
- i. Accounting Integration (High-Level)
- j. Implementation Roadmap (0 → Go-Live)
- k. Infographic Script (for design)

0. Overview

0.1 Goal

Build a **professional ERP product** that:

- Runs **multiple companies** on a single server and single codebase.
- Supports **multiple business activities** (manufacturing, trading, retail, services, real estate, cars, academies, etc.).
- Allows **per-company customization** (modules, costing model, currencies, units, reports).
- Is implemented with a clear, staged roadmap to reduce errors and rework.

0.2 Target Business Types

The system should be able to support at least:

- Manufacturing factories.
- Trading (wholesale / retail).
- Supermarkets.
- Pharmacies.
- Clothing shops.
- Restaurants & cafes.
- Contracting companies.
- Real estate (sale / renting of units).
- Car dealerships / renting.
- Academies & training centers.

The ERP core is shared; each client company uses a **profile** based on its activity type.

1. Supported Business Types & Customization

1.1 Activity Type per Company

Each company has an `activity_type`, e.g.:

- `manufacturing`
- `trading`
- `supermarket`
- `pharmacy`
- `clothing`
- `restaurant`
- `cafe`
- `academy`
- `contracting`
- `real_estate`
- `cars`
- `mixed`

1.2 Per-Client Customization Flow

When you onboard a new client company:

1. Create a **company record** with:
 - Name, code, base currency, activity type.

2. Pick an **activity template**:

- Enables relevant modules (e.g. manufacturing, inventory, costs, accounting).
- Chooses default costing model(s) (product, work-order/job, project).
- Suggests typical units of measure.
- Suggests key reports & dashboards.

3. Optionally **override**:

- Enabled modules.
- Costing behavior.
- Units & currencies.

The system must be **dynamic** enough to support custom combinations of modules per company.

2. Tech Stack & Architecture (incl. Docker)

2.1 Tech Stack

- **Backend**: Python
 - Django (monolith) or FastAPI for APIs.
- **Database**: PostgreSQL.
- **Frontend**: Web admin/dashboard UI.
 - HTML/CSS/Bootstrap or an admin template (AdminLTE, SB Admin, etc.).
- **Reverse Proxy**: Nginx.
- **Cache / Queue (optional)**: Redis.
- **Containerization**: Docker + docker-compose.

2.2 Docker Architecture

`docker-compose.yml` should define:

- `web`:
 - Backend container (Python app).
 - Environment variables for DB, secrets, debug flag, etc.
- `db`:
 - PostgreSQL container with a persistent volume.
- `nginx`:
 - Reverse proxy for HTTP/HTTPS.
- `redis` (optional, later phase):
 - For caching, sessions, or background jobs.

-

Early steps:

1. Create repo & Python environment.
 2. Write `Dockerfile` for backend.
 3. Write `docker-compose.yml` (web, db, nginx, optionally redis).
 4. Implement a simple `/health` endpoint and run everything locally via Docker.
-

3. Multi-Company, Users & Permissions

3.1 Companies & Branches

Tables:

- `companies`
 - `id` (PK)
 - `name`
 - `code`
 - `base_currency` (e.g. EGP, USD)
 - `activity_type`
 - `is_active`
- `branches`
 - `id`
 - `company_id` (FK → `companies`)
 - `name`
 - `code`
 - `address` , `phone`
- `company_settings`
 - `company_id` (PK/FK)
 - `enabled_modules` (JSON array)
 - `cost_method` (e.g. `"weighted_average"`)
 - `has_manufacturing` (bool)
 - `has_projects` (bool)
 - `has_pos` (bool)
 - `pos_style` (retail / restaurant / pharmacy / etc.)

3.2 Users & Permissions

Tables:

- `users`
 - `id`, `email`, `password_hash`
 - `full_name`
 - `is_active`
- `roles`
 - `id`
 - `company_id`
 - `name` (Admin, Sales Manager, POS Cashier, Accountant, HR, etc.)
- `permissions`
 - `id`
 - `code` (e.g. `sales.view_invoice`, `inventory.adjust_stock`, etc.)
- `user_roles`
 - `user_id`
 - `company_id`
 - `branch_id` (optional)
 - `role_id`
- `role_permissions`
 - `role_id`
 - `permission_id`

Key rule: all business queries must be filtered by `company_id` (and `branch_id` where applicable). Middleware should enforce this.

4. Master Data (Core Business Data)

4.1 Partners (Customers & Suppliers)

- `partners`
 - `id`, `company_id`
 - `type` (`customer`, `supplier`, `both`)
 - `name`, `code`
 - `phone`, `email`, `address`
 - `tax_number`
 - `opening_balance`
 - `credit_limit`

- `is_active`

4.2 Items & Categories

- `item_categories`
 - `id`, `company_id`
 - `name`, `code`
 - `parent_id` (for tree structure)
- `items`
 - `id`, `company_id`
 - `name`, `code`, `barcode`
 - `item_category_id`
 - `base_unit_id`
 - `sale_unit_id` (optional)
 - `purchase_unit_id` (optional)
 - `type` (`stock`, `service`, `manufactured`)
 - `default_sale_price`
 - `default_purchase_price`
 - `min_stock_level`
 - `expiry_control` (bool; important for pharmacies)
 - `attributes` (JSON, e.g. size/color for clothing)

4.3 Warehouses & Stock

- `warehouses`
 - `id`, `company_id`
 - `branch_id`
 - `name`, `code`, `address`
- `warehouse_stock`
 - `warehouse_id`
 - `item_id`
 - `quantity`
 - `average_cost`
- `inventory_movements`
 - `id`, `company_id`
 - `item_id`

- `warehouse_from_id`, `warehouse_to_id`
- `quantity`
- `movement_type`
- `unit_cost`
- `total_cost`
- `document_type`, `document_id`

5. Currencies & Units of Measure

5.1 Multi-Currency & FX

- `currencies`
 - `code`, `name`
- `currency_rates`
 - `id`
 - `currency_code`
 - `rate_to_base`
 - `valid_from` (timestamp)

In each financial document (e.g. invoice):

- `document_currency`
- `fx_rate_used`
- `total_in_document_currency`
- `total_in_base_currency`

When a receivable/payable is settled, compute FX differences between:

- Rate at document time, and
- Rate at payment/settlement time,

and post the gain/loss to dedicated FX gain/loss accounts (later in accounting integration).

5.2 Units & Conversions

- `units`
 - `id`, `company_id` (or global)
 - `name` (Piece, Box, Kg, Meter, Litre, etc.)
 - `symbol`

- `unit_type` (weight, length, count, volume)
- `unit_conversions`
 - `id`
 - `from_unit_id`
 - `to_unit_id`
 - `factor` (e.g. `1 box = 12 pieces`)

Each item links to:

- Base unit.
- Optional sales unit / purchase unit.
- Conversion factor where applicable.

6. Functional Modules

6.1 Sales

Tables:

- `sales_invoices`
- `sales_invoice_lines`
- `sales_payments`

Key behaviors:

- Confirming a sales invoice:
 - Creates stock-out `inventory_movements` from selected warehouse.
 - Uses `average_cost` to compute cost of goods sold.
 - Later: posts accounting journal entries.

Frontend pages:

- Sales Dashboard (KPIs + charts).
- Customers list (search, filters).
- Sales invoices list with filters:
 - Date range, customer, branch, status, currency.
- Sales invoice form:
 - Header (company, branch, warehouse, customer, currency, FX).
 - Lines (item, qty, unit, price, discount, tax, total).

- Summary (subtotal, discount, tax, grand total, paid, remaining).

6.2 Purchases

Tables:

- `purchase_invoices`
- `purchase_invoice_lines`
- `purchase_payments`

Behavior:

- Confirming a purchase invoice:
 - Creates stock-in `inventory_movements` of type `purchase_in`.
 - Updates `average_cost` per item/warehouse.

Frontend pages:

- Suppliers list.
- Purchase invoices list + filters.
- Purchase invoice form.

6.3 POS

Used for supermarkets, pharmacies, clothing stores, restaurants, cafes, etc.

Tables:

- `pos_sessions`
- `pos_orders`
- `pos_order_lines`

Behavior:

- Each session tracks cash in/out for a cashier.
- Each order may link to a sales invoice (for full back-office traceability).

Frontend pages:

- Full-screen POS interface:
 - Product grid, search by barcode/name, cart, payment, print.
- POS sessions:
 - Open/close session, opening/closing balances, variances.

6.4 Inventory Module

Pages:

- Items list & form.
- Stock movements.
- Stock balance.
- Stock taking & adjustments.

Filters:

- Company, branch, warehouse, date range, item, category, movement type.

6.5 Manufacturing (Factories)

Tables:

- `boms` , `bom_lines`
- `work_orders`
- `work_order_consumption` (raw materials)
- `work_order_labor` (labor hours)
- `work_order_overheads` (e.g. electricity, rent)
- `work_order_output` (finished goods)

Behavior:

- A work order:
 - Consumes raw materials from warehouses (stock-out movements).
 - Records labor and overhead costs.
 - Produces finished goods as stock-in with computed `cost_per_unit` .

6.6 Projects / Contracting / Real Estate / Cars

Tables:

- `projects` / `jobs`
- `job_materials`
- `job_labor`
- `job_overheads`

Behavior:

- Each project/job accumulates costs (materials, labor, overheads).
- Profitability reports compare total revenue vs total cost per job/project.

6.7 HR (Optional)

Tables:

- `employees` , `departments`

- `attendance_records`
- `leave_requests`
- `payroll_runs` , `payroll_lines`

6.8 Accounting (General Ledger)

Tables:

- `chart_of_accounts`
- `journal_entries`
- `journal_lines`

Reports:

- Trial Balance.
- Income Statement.
- Balance Sheet.

6.9 Reports & Dashboards

For each module, define:

- Dashboards (KPIs, charts).
- Detailed reports with flexible filters.

Examples:

- Sales by date, customer, item, branch, currency.
- Purchases by supplier, item.
- Inventory aging, slow-moving items, stock valuation.
- HR attendance & payroll.
- Costs by product/project.
- Financial statements.

7. Costing Models

7.1 Product-Level Costing (Trading / Retail)

- Use **Weighted Average** as inventory costing method.
- Maintain, per `(company, warehouse, item)` :
 - `on_hand_qty`
 - `average_cost`
- On stock-in (purchases, adjustments, production):

- Update `average_cost` and `on_hand_qty`.
- On stock-out (sales, consumption):
 - Use `average_cost` as unit cost for COGS or consumption.

7.2 Work-Order / Manufacturing Costing

Cost of finished goods per work order:

```
Finished Cost = Raw Materials Cost
               + Direct Labor Cost
               + Overhead Allocation
```

Where:

- Raw materials cost uses weighted average at time of consumption.
- Labor cost = hours × hourly rate.
- Overheads = allocated amount (electricity, rent, etc.).

7.3 Job/Project Costing

For contracting, real estate, car renting, etc.:

```
Project Cost = Materials + Labor + Overheads
Profit       = Revenue   - Project Cost
```

8. Accounting Integration (High-Level)

Accounting is implemented in a later phase, after stabilizing business flows.

High-level examples:

- Sales invoice:
 - Debit: Accounts Receivable / Cash.
 - Credit: Sales Revenue.
 - Credit/Debit: VAT accounts as needed.
 - Debit: Cost of Goods Sold.
 - Credit: Inventory.
- Purchase invoice:
 - Debit: Inventory.
 - Debit: Input VAT.
 - Credit: Accounts Payable.
- Manufacturing completion:

- Debit: Finished Goods Inventory.
- Credit: Raw Materials Inventory.
- Credit: WIP/Cost pools for labor and overheads as configured.
- FX differences:
 - Post gains/losses to FX gain/loss accounts when rates change between document and settlement dates.

9. Implementation Roadmap (0 → Go-Live)

Follow this order to reduce risk and avoid logical errors.

Phase 0 – Infrastructure

1. Set up repo and Python environment.
2. Implement `Dockerfile` and `docker-compose.yml` (web, db, nginx).
3. Create health check endpoint and test full stack locally.

Phase 1 – Multi-Company Core & Auth

1. Implement models: `users`, `companies`, `branches`.
2. Implement authentication (login/logout, password hashing).
3. Implement a middleware/service to enforce `company_id` scoping on all business requests.

Phase 2 – Roles & Permissions (Basic)

1. Implement `roles`, `permissions`, `user_roles`, `role_permissions`.
2. Add simple permission checks on key endpoints (e.g., only sales users can manage invoices).

Phase 3 – Master Data

1. Implement `partners` (customers/suppliers).
2. Implement `item_categories`, `units`, `items`.
3. Implement `warehouses`, `warehouse_stock`, `inventory_movements`.
4. Build basic CRUD UIs for these entities.

Phase 4 – Currencies & Units

1. Implement `currencies` and `currency_rates`.
2. Add currency fields to financial documents.
3. Implement `units` and `unit_conversions` and link to `items`.

Phase 5 – Sales & Purchases

1. Implement `sales_invoices`, `sales_invoice_lines`, `sales_payments`.
2. Implement `purchase_invoices`, `purchase_invoice_lines`, `purchase_payments`.

3. Implement confirm logic:
 - Create inventory movements on confirm.
 - Compute COGS using weighted average.
4. Build invoice UIs and listing pages with filters.

Phase 6 – Inventory Reporting

1. Stock balance report.
2. Stock movement report.
3. Stock taking & adjustment flows.

Phase 7 – Manufacturing & Projects (As Needed)

1. Implement BOMs & Work Orders.
2. Implement raw material consumption and finished goods production.
3. Implement `projects/jobs` and cost tracking for contracting/real_estate/cars as needed.

Phase 8 – HR (Optional)

1. Implement employees & departments.
2. Implement attendance & leave.
3. Implement basic payroll if required.

Phase 9 – Accounting Integration

1. Implement `chart_of_accounts`, `journal_entries`, `journal_lines`.
2. Wire business events (sales, purchases, production, FX differences) to journal posting logic.
3. Implement core financial reports (Trial Balance, P&L, Balance Sheet).

Phase 10 – Costs & Dashboards

1. Build product cost and project cost reports.
2. Build dashboards per module (Sales, Inventory, Manufacturing, Projects, Finance).

Phase 11 – Hardening & Production

1. Add indexes & tune performance.
2. Harden security (permissions, logging, audit trails).
3. Implement backup & restore strategy.
4. Deploy to production server (via Docker).

10. Infographic Script (for Design)

Use this as a guide to design a one-page infographic (image or PDF).

Layout suggestion

1. Header (top):

- Title: "Flexible Multi-Activity ERP System".
- Sub-title: "One Core – Many Business Types".

2. Core Stack Box (top-left):

- Icons: Python, PostgreSQL, Docker, Nginx.
- Text bullets:
 - Backend: Python (Django/FastAPI).
 - Database: PostgreSQL.
 - Web Dashboard UI.
 - Docker Deployment.

3. Multi-Company & Activities (top-right):

- 3–4 circles labeled Company A, B, C.
- Under each, small icons (factory, pharmacy, supermarket, shop, building, car, graduation cap).
- Arrow/text: "One Core – Configurable per Activity".

4. ERP Modules Ring (center):

- Central circle: "ERP Core".
- Around it, smaller circles:
 - Sales
 - Purchases
 - POS
 - Inventory
 - Manufacturing
 - HR
 - Accounting
 - Costs
 - Reports & Dashboards

5. Company Onboarding Flow (middle-left):

- Vertical or horizontal flow with 5 steps:
 - i. Create Company & Branches.
 - ii. Choose Activity Type.
 - iii. Enable Modules & Cost Models.
 - iv. Configure Currencies & Units.

- v. Go Live (Sales, Purchases, POS, Production, Projects).

6. **Data & Costing (middle-right):**

- Data Core box:
 - Items, Partners, Warehouses, Projects, Employees.
- Costing box:
 - Product Costing (Weighted Average).
 - Job/Project & Manufacturing Costing.
 - FX Differences.

7. **Frontend & Reports (bottom-left):**

- Illustration of a dashboard: KPI cards, a chart, a table with filters sidebar.
- Caption: "Customizable Filters & Reports per Module".

8. **Infrastructure & Deployment (bottom-right):**

- Docker containers drawn as boxes: Web, DB, Nginx, Redis.
- Caption: "Single Server – Multi-Company – Easy to Deploy & Scale".

ERP ERP System Infographic Description

ERP System Infographic – Design Reference

Use this page as the blueprint for a single-page infographic that visually explains the ERP product.

Layout Overview

- Orientation: A4 or 16:9 landscape.
- Style: Clean, modern, flat icons, muted professional colors.
- Sections are arranged in a logical flow from top (concept) to bottom (implementation).

Sections

1. Header (Top)

- Title: **Flexible Multi-Activity ERP System**
- Sub-title: **One Core – Many Business Types**
- Optional: Short tagline like “Factories • Retail • Services • Real Estate • Cars • Academies”.

2. Core Stack (Top-Left)

- A box or card with icons for:
 - Python
 - PostgreSQL
 - Docker
 - Nginx
- Bullet text inside the box:
 - *Backend: Python (Django/FastAPI)*
 - *Database: PostgreSQL*
 - *Web Dashboard UI*
 - *Docker Deployment*

3. Multi-Company & Activities (Top-Right)

- 3–4 circles labeled **Company A, Company B, Company C**.
- Below or near each circle, place a small activity icon:
 - Factory (manufacturing)
 - Shopping cart / supermarket
 - Pill bottle (pharmacy)

- T-shirt (clothing)
- Restaurant plate / coffee cup
- Building (real estate)
- Car (cars)
- Graduation cap (academy)
- Add an arrow or label: **“One Core – Configurable per Activity”**.

4. ERP Modules Ring (Center)

- Central circle: **ERP Core**.
- Surround it with smaller circles for each module:
 - Sales
 - Purchases
 - POS
 - Inventory
 - Manufacturing
 - HR
 - Accounting
 - Costs
 - Reports & Dashboards
- Connect each small circle to the center to show they are all powered by the same core.

5. Company Onboarding Flow (Middle-Left)

A 5-step process drawn as a vertical or horizontal flow:

1. **Create Company & Branches**
2. **Choose Activity Type**
3. **Enable Modules & Cost Models**
4. **Configure Currencies & Units**
5. **Go Live** (Sales, Purchases, POS, Production, Projects)

Use arrows between each step with short labels.

6. Data & Costing (Middle-Right)

Two neighboring boxes:

- **Data Core**
 - Items

- Partners
- Warehouses
- Projects / Jobs
- Employees
- **Costing**
 - Product Costing (Weighted Average)
 - Job / Project Costing
 - Manufacturing Costing (Raw + Labor + Overheads)
 - FX Differences

Connect these boxes visually to the modules ring to show that all modules depend on the core data & costing.

7. Frontend & Reports (Bottom-Left)

- Draw a stylized dashboard screen:
 - Top: 3–4 KPI cards (e.g., Total Sales, Stock Value, Open Projects).
 - Middle: A line or bar chart.
 - Bottom: A data table with a filter sidebar.
- Caption: **“Customizable Filters & Reports per Module”**.

8. Infrastructure & Deployment (Bottom-Right)

- Show Docker containers as rectangles labeled:
 - `Web`
 - `DB`
 - `Nginx`
 - `Redis` (optional)
- Place them on top of a “Server” or “Cloud” icon.
- Caption: **“Single Server – Multi-Company – Easy to Deploy & Scale”**.

This structure should be enough for a designer (or you) to create a clean infographic as PNG/JPEG or PDF.

ERP README – Checklist Version

ERP Product – Execution Checklist

Use this as a high-level checklist while coding and deploying the ERP system.

Phase 0 – Infrastructure

- ☐ Initialize repo & Python environment.
- ☐ Create `Dockerfile` for backend.
- ☐ Create `docker-compose.yml` for `web`, `db`, `nginx` (and `redis` later).
- ☐ Implement `/health` endpoint.
- ☐ Run the full stack locally via Docker.

Phase 1 – Multi-Company Core & Auth

- ☐ Implement models: `users`, `companies`, `branches`.
- ☐ Implement authentication (login/logout, password hashing).
- ☐ Implement company selection & active `company_id` per session.
- ☐ Add middleware/service to enforce `company_id` scoping.

Phase 2 – Roles & Permissions (Basic)

- ☐ Implement `roles`, `permissions`, `user_roles`, `role_permissions`.
- ☐ Protect key endpoints with basic permission checks.

Phase 3 – Master Data

- ☐ Implement `partners` (customers/suppliers).
- ☐ Implement `item_categories`, `units`, `items`.
- ☐ Implement `warehouses`, `warehouse_stock`, `inventory_movements`.
- ☐ Build simple CRUD UIs for these entities.

Phase 4 – Currencies & Units

- ☐ Implement `currencies`, `currency_rates`.
- ☐ Add currency fields to financial documents.
- ☐ Implement `units`, `unit_conversions`.
- ☐ Link units & conversions to `items` and document lines.

Phase 5 – Sales & Purchases

- ☐ Implement `sales_invoices`, `sales_invoice_lines`, `sales_payments`.
- ☐ Implement `purchase_invoices`, `purchase_invoice_lines`, `purchase_payments`.
- ☐ Implement "confirm invoice" logic (stock in/out + cost calculation).
- ☐ Build invoice forms and listing pages with filters.

Phase 6 – Inventory Reporting

- ☐ Implement stock balance report.
- ☐ Implement stock movement report.
- ☐ Implement stock taking and adjustment flows.

Phase 7 – Manufacturing & Projects (as needed)

- ☐ Implement `boms`, `bom_lines`, `work_orders`.
- ☐ Implement `work_order_consumption`, `work_order_output` (and labor/overheads if needed now).
- ☐ Implement `projects/jobs`, `job_materials`, `job_labor`, `job_overheads`.
- ☐ Implement cost reports for products and jobs/projects.

Phase 8 – HR (Optional)

- ☐ Implement `employees`, `departments`.
- ☐ Implement `attendance_records`, `leave_requests`.
- ☐ Implement `payroll_runs`, `payroll_lines` if required.

Phase 9 – Accounting Integration

- ☐ Implement `chart_of_accounts`, `journal_entries`, `journal_lines`.
- ☐ Connect business events (sales, purchases, production, FX differences) to journal posting.
- ☐ Implement Trial Balance, P&L, Balance Sheet.

Phase 10 – Costs & Dashboards

- ☐ Implement product cost reports.
- ☐ Implement job/project cost reports.
- ☐ Implement dashboards for Sales, Inventory, Manufacturing, Projects, Finance.

Phase 11 – Hardening & Production

- ☐ Add DB indexes, tune queries.
- ☐ Harden security: permissions, logging, audit trails.
- ☐ Implement backup & restore strategy.

☐ Deploy to production server (via Docker).

ERP System – Arabic High-Level Summary

مقدمة النظام (Introduction) ERP

نظام ERP (Enterprise Resource Planning) هو نظام معلوماتي متكامل يدير العمليات التجارية للشركة.

أهداف النظام (Objectives)

يهدف النظام ERP إلى تحسين الكفاءة التشغيلية، تقليل التكاليف، وزيادة الإنتاجية.

- تحسين إدارة الموارد البشرية والمالية (Human Resource Management) وتقليل التكاليف التشغيلية.
- تحسين إدارة المخزون والمشتريات (Inventory Management).
- تحسين إدارة العمليات الإنتاجية (Manufacturing) وتقليل الهدر.

المكونات الرئيسية (Main Components)

- إدارة المبيعات (Sales)
- إدارة المشتريات (Purchases)
- نقاط البيع (POS)
- إدارة المخزون (Inventory)
- التصنيع (Manufacturing)
- إدارة المشاريع/الوظائف (Projects/Jobs)
- إدارة الموارد البشرية (HR) – إدارة الموظفين
- المحاسبة (Accounting)
- التكاليف (Costs)
- لوحات التحكم Dashboards

المزايا (Benefits)

- **Multi-Company:** إدارة عدة شركات من خلال نظام واحد.
- **Multi-Activity:** إدارة أنشطة الشركة المختلفة.
- **Multi-Currency:** دعم عملات متعددة.
- **Advanced Costing:** نظام متقدم لتكاليف الإنتاج.
- **Docker:** دعم بيئة الحاويات.

الخطوات (Steps)

1. إعداد النظام واختبارها.
2. تدريب المستخدمين.

3. `__init__` 方法中 `self` 参数指向实例对象。
4. `__str__` 方法返回的字符串是对象在内存中的地址。
5. (...`__str__` 方法返回的字符串是对象在内存中的地址) 方法返回的字符串是对象在内存中的地址。

__str__ 方法

`__str__` 方法 (即 `__str__` + `__str__` + `__str__`) 方法返回的字符串是对象在内存中的地址。

- `__str__` 方法返回的字符串是对象在内存中的地址。
- `__str__` 方法返回的字符串是对象在内存中的地址。
- `__str__` 方法返回的字符串是对象在内存中的地址。

README.md – Raw Copy for Local Project

This page mirrors the README content so you can easily copy it into a local [README.md](#) file in your project.

```
# ERP Product - Full Specification & Roadmap
```

```
> This README is the main reference document for designing and implementing a flexible, multi-activity, multi-company ERP system. Follow the phases at the end in order to minimize errors and avoid architectural rework.
```

```
## Table of Contents
```

- 0. Overview
- 1. Supported Business Types & Customization
- 2. Tech Stack & Architecture (incl. Docker)
- 3. Multi-Company, Users & Permissions
- 4. Master Data (Core Business Data)
- 5. Currencies & Units of Measure
- 6. Functional Modules
- 7. Costing Models
- 8. Accounting Integration (High-Level)
- 9. Implementation Roadmap (0 → Go-Live)
- 10. Infographic Script (for design)

```
... (same full content as in the README Version page) ...
```

ERP System – Infographic Preview (Text Layout)

This page gives a compact bullet-point version of the infographic you will see as an image.

- Header: Flexible Multi-Activity ERP System
- Sub: One Core – Many Business Types
- Top-left: Tech stack (Python, PostgreSQL, Docker, Nginx)
- Top-right: Multi-company & activities icons (factory, pharmacy, supermarket, clothing, restaurant, building, car, academy)
- Center: ERP Core circle with modules ring (Sales, Purchases, POS, Inventory, Manufacturing, HR, Accounting, Costs, Reports & Dashboards)
- Mid-left: 5-step onboarding flow (Create Company → Choose Activity → Enable Modules & Cost Models → Configure Currencies & Units → Go Live)
- Mid-right: Data & Costing boxes (Data Core + Costing types)
- Bottom-left: Dashboard UI mock (cards + chart + table + filters)
- Bottom-right: Docker infrastructure (Web, DB, Nginx, Redis) over a server/cloud icon

The generated image will follow this layout approximately.

NOTE: How to export this README and infographic

To get the two files you asked for (README + infographic) on your machine:

1. Open the doc Private (<https://app.clickup.com/90152588457/docs/2kyr6n59-535>).
2. Use ClickUp's **Export / Print** options to export:
 - The page "**ERP Product – Full Specification & Roadmap (README Version)**" as a PDF (this is your README file) or copy it into `README.md` in your code repo.
 - The page "**ERP ERP System Infographic Description**" as a PDF, or give it to a designer to convert to a designed infographic image.
3. Optionally, use the **Checklist Version** page as a quick implementation to-do list.

This note is just for your reference; delete it anytime.

Untitled